

Design Document Overview

Roll no.: IMT2023051

1. Introduction

This project involves managing events, venues, and congregations, focusing on scheduling, reservation management, and event handling with specific constraints, such as time alignment and minimum event duration.

2. Core Classes & Structs

The **Date** and **Time** classes are responsible for handling date and time validation, manipulation, and comparison. The ``Date`` class stores day, month, and year, with methods for validation, checking adjacency of two dates, and performing comparisons. The ``Time`` class stores hour and minute, offering methods for validation and calculating time differences.

The **Event** struct represents an event with attributes like event name, congregation name, date, start, and end times. It includes methods for validating time gaps between events. The **Reservation** struct represents a venue reservation by a congregation, with start and end dates, organized in a linked list to manage reservations chronologically.

The **Congregation** class manages congregation data, including reservations, and provides methods to add, show, and remove reservations. The **Venue** class manages venue data and handles event scheduling, reservation management, and conflict checking.

CongregationManager and **VenueManager** classes manage collections of congregations and venues, offering methods to add, remove, and query them. The **Calendar** class integrates these managers, providing high-level functions like adding congregations or venues, reserving or freeing venues, and managing events. Finally, the **Decoder** struct parses user commands, delegating operations to the ``Calendar`` class while ensuring valid input formats for dates, times, and locations.

3. Command Functions

Venue management functions include ``addVenue``, ``delVenue``, and ``showVenues``. Congregation management functions encompass ``addCongregation``, ``delCongregation``, and ``showCongregations``. Reservation management involves ``reserveVenue``, ``freeVenue``, and ``showReserved``. Event management is handled by ``addEvent``, ``delEvent``, ``showEvents``, and ``showCalendar``.

4. Error Handling

Error handling is facilitated by the **Status Enum**, which represents various status codes such as ``OK``, ``DUPLICATE_VENUE``, ``NO_EVENT``, and ``TIME_CONFLICT``. The ``printCode`` method maps these status codes to user-friendly error messages.

5. Design Principles

The design emphasizes **encapsulation**, with each class and struct encapsulating specific responsibilities to promote modular design. **Linked lists** are used for managing reservations and events in chronological order. The design also prioritizes stringent validation of dates, times, and reservations to avoid conflicts.

6. Constraints

The project enforces strict time validation, ensuring that events align with 15-minute intervals and have a minimum duration of 30 minutes. Reservation overlap is prevented, disallowing multiple reservations for the same venue and congregation. Additionally, the design includes checks for date adjacency in event scheduling to ensure continuity.

7. Assumptions

The design assumes that input strings for dates, times, and locations are always valid, with no external date format validation required. This assumption simplifies input processing and ensures that the focus remains on managing the core functionalities of the system.

UML Diagram

